

ADVANCES IN CONTINUAL GRAPH LEARNING FOR ANTI-MONEY LAUNDERING SYSTEMS: A COMPREHENSIVE REVIEW

A PREPRINT

Bruno Deprez*
KU Leuven
University of Antwerp - imec

Wei Wei
University of Antwerp - imec, IDLab

Wouter Verbeke
KU Leuven

Bart Baesens
KU Leuven
University of Southampton

Kevin Mets
University of Antwerp - imec, IDLab

Tim Verdonck
University of Antwerp - imec
KU Leuven

April 1, 2025

ABSTRACT

Financial institutions are required by regulation to report suspicious financial transactions related to money laundering. Therefore, they need to constantly monitor vast amounts of incoming and outgoing transactions. A particular challenge in detecting money laundering is that money launderers continuously adapt their tactics to evade detection. Hence, detection methods need constant fine-tuning. Traditional machine learning models suffer from catastrophic forgetting when fine-tuning the model on new data, thereby limiting their effectiveness in dynamic environments. Continual learning methods may address this issue and enhance current anti-money laundering (AML) practices, by allowing models to incorporate new information while retaining prior knowledge. Research on continual graph learning for AML, however, is still scarce. In this review, we critically evaluate state-of-the-art continual graph learning approaches for AML applications. We categorise methods into replay-based, regularization-based, and architecture-based strategies within the graph neural network (GNN) framework, and we provide in-depth experimental evaluations on both synthetic and real-world AML data sets that showcase the effect of the different hyperparameters. Our analysis demonstrates that continual learning improves model adaptability and robustness in the face of extreme class imbalances and evolving fraud patterns. Finally, we outline key challenges and propose directions for future research.

Keywords Continual Learning · Anti-Money Laundering · Graph Neural Networks · Fraud Detection · Catastrophic Forgetting

1 Introduction

Criminal enterprise activities generate income streams that cannot be directly used because of their illegal origins. Therefore, criminals *launder money* to make illicitly obtained funds appear legitimate [25]. The United Nations Office on Drugs and Crime (UNODC) [54] has estimated that an amount equal to about 2% to 5% of global GDP is laundered each year, amounting to USD 2 trillion. This money is used to expand criminal activity and to finance terrorism [25], resulting in enormous socioeconomic pressure.

Generally, money laundering approaches involve three main steps [54, 25]. During **placement**, the illegal money enters the financial system, often in jurisdictions where regulation and enforcement are less strict. **Layering** involves mixing the illegal funds with legitimately obtained money across multiple transactions. This obscures the initial source of the money, making it harder to uncover the illegal origin. At **integration**, the money is spent on legitimate purchases. After

*Corresponding author: bruno.deprez@kuleuven.be

completing this final step, the money is successfully laundered. Actual money laundering approaches may also include fewer or more steps.

The framework given above illustrates that money laundering involves many payments over multiple accounts, warranting the use of network analytics [47]. A popular way of adopting network analytics is via graph neural networks (GNNs). GNNs have shown promising results for fighting financial crime [42] with adoption in credit card fraud detection [55, 57, 56], insurance fraud detection [44, 11], and anti-money laundering [10]. GNNs are able to detect fraudulent behaviour by learning complex, non-linear patterns in network data [42].

Financial networks and fraud characteristics evolve over time, necessitating the fine-tuning of these GNNs when new data comes in. This fine-tuning can cause the model to suffer from catastrophic forgetting [15, 8, 9], where learning sequentially on *new* data while discarding *old* data leads to significant performance loss on earlier observations.

Continual learning, also known as incremental learning or lifelong learning, aims to mitigate the problem of catastrophic forgetting [16]. Research in this field typically aims to adapt regular deep learning methods or develop new methods that are able to accumulate and consolidate knowledge. One of the key assumptions that underlie continual learning, is that data is no longer (fully) accessible after models have been trained.

Continual learning is key for effective and dynamic anti-money laundering (AML). Financial institutions often face enormous transaction volumes that require continuous monitoring. However, they face computational constraints when implementing AML methods in practice. First, there is limited computing power and budget to update AML models, making periodical retraining from scratch impractical. Second, there are regulatory constraints on how much data can be stored and for how long. Finally, money laundering methods, as for other types of fraud, are evolving constantly [5, 59], so the distribution of illegitimate transactions changes over time. However, when training to detect these new tactics, models should be able to retain knowledge about old ones, in case these are used again. Otherwise, fraudsters could just rotate between tactics to evade detection.

Continual learning performs well under these constraints. First, it updates existing models, so limited additional training is required. Second, updating the model can be done using only the most recent data, so there is no need to store all historical data indefinitely. Finally, continual learning is specifically designed to retain previous knowledge when learning from new data, so older *modi operandi* should still be detected.

Despite all this, research on continual graph learning for AML is rare. Furthermore, current experiments and benchmarks in literature lack an in-depth discussion on the effect of the many choices underlying the applied continual learning framework. Therefore, this work sets out to answer the following research questions:

- RQ1** What is the current state of the literature on continual graph learning for anti-money laundering?
- RQ2** What is the impact of the hyperparameters of the GNN and continual learning methods on performance and forgetting?
- RQ3** What is the impact of depth and width of the GNN on performance and forgetting?
- RQ4** Which methods are best suited to overcome catastrophic forgetting for anti-money laundering?

To answer these questions, we conduct an in-depth literature review, summarising the current research on GNNs for AML, continual learning for financial fraud detection, and previous work on the effect of the involved hyperparameters. This review of the literature is complemented by an extensive experimental study to analyse the performance and forgetting of AML network methods on two open-source data sets. The contributions of our work are, hence, as follows:

- We present an in-depth review of the current state-of-the-art in continual graph learning for fraud detection;
- We introduce and investigate the implications of continual graph learning on anti-money laundering, for edge as well as node classification;
- We present the result of extensive experiments on two open-source AML data sets and analyse the effects of various choices on performance.

The code of the presented experiments is publicly available on [github](https://github.com/VerbekeLab)² to facilitate peer researchers and practitioners to replicate and extend the reported results.

The remainder of this paper is organised as follows. Preliminary theory on graphs, graph neural networks and continual learning is discussed in Section 2. A review of the literature is presented in Section 3. Section 4 presents the experimental methodology, with results and discussion provided in Section 5. Section 6 concludes this work and presents directions for future research.

²<https://github.com/VerbekeLab>

2 Preliminaries

2.1 Graphs

A graph $G(V, E)$ is defined via two sets, V and E . The elements of set $V = \{v_1, \dots, v_n\}$ represent the nodes in the graph, while set $E \subset V \times V$ represents the edges that connect the nodes. An edge between node i and j is denoted as e_{ij} . In this work, we consider homogeneous networks. It is assumed that nodes are assigned a vector $x_i \in \mathbb{R}^m$ with feature values. The matrix containing all feature vectors is denoted by $X \in \mathbb{R}^{n \times m}$.

2.2 Graph Neural Networks

The initial idea of deep learning on graphs was introduced by Scarselli et al. [50] and Scarselli et al. [51], based on message passing. The idea of message passing is still present in GNNs, where a node's representation is updated iteratively based on the node's neighbours.

Formally, given a graph $G(V, E)$, graph neural networks (GNNs) construct the representation of node i at layer l , denoted by $h_i^{(l)}$, as

$$h_i^{(l)} = \phi^{(l-1)} \left(h_i^{(l-1)}, \sum_j \hat{A}_{ij} \psi^{(l-1)} \left(h_i^{(l-1)}, h_j^{(l-1)} \right) \right), \quad (1)$$

where $\phi^{(l)}$, and $\psi^{(l)}$ are layer-dependent functions, and $\hat{A}_{ij}$ is the normalized adjacency matrix, including self-loops. Most of the time, the initial embedding is set equal to the node features, $h_i^{(0)} = x_i$.

The most widely adopted graph neural networks are Graph Convolutional Networks (GCN) [20], Graph SAmple and aggreGatE (GraphSAGE) [17], Graph ATtention network (GAT) [60] and Graph Isomorphism Networks (GIN) [67]. GCNs introduced by Kipf and Welling [20] aggregate neighbourhood information based on convolutions. Hamilton et al. [17] extends on this idea by introducing GraphSAGE resulting into an inductive method. Veličković et al. [60] introduces attention mechanisms using GAT, allowing the distinction between important and less important neighbours in the network. Finally, Xu et al. [67] introduced GIN, relying on the Weisfeiler-Lehman graph isomorphism test to come to a more versatile version of GNNs.

2.3 Continual Learning

In continual learning, a model needs to sequentially learn disjoint tasks $\mathcal{T} = \{\mathcal{T}_1, \dots, \mathcal{T}_K\}$ [21, 1]. Specific observations are provided with each task $\mathcal{T}_i$, while access to data of previous tasks is often limited or even prohibited. Each task has its corresponding feature set X_i , and task-specific label $y_i \in \mathcal{Y}_i$, with label set $\mathcal{Y}_i = \{y^1, \dots, y^{c_i}\}$, where c_i represents the number of classes in task $\mathcal{T}_i$. Sometimes, a specific assumption is made that task data is provided as $(\mathcal{X}, \mathcal{Y}, \mathcal{D}_C)$ with $\mathcal{D}_C$ the underlying distribution, also called context set [9].

Depending on the available information and the format of the tasks provided, four different continual learning settings are discerned, i.e., task-incremental, domain-incremental, class-incremental and time-incremental learning [58, 22]. The first three are well-established in continual learning [58]. **Task-incremental learning** consists of a sequence of distinct tasks to be learned, where the model knows which task is currently presented, even at test time. **Domain-incremental learning** describes the scenario in which the problem is the same, but the distribution of the tasks shifts. Here, no information about the task is provided at test time. In **class-incremental learning**, a growing number of classes are provided with each new task, but no task information is provided. Hence, the methods should also be able to learn to distinguish the current task that is provided. **Time-incremental learning** encompasses problems where data is provided in streaming format, and where the distribution might shift over time. Some work considers this to be a separate setting [22], while it can also be seen as a specific case of domain-IL.

To mitigate catastrophic forgetting, different methods have been developed, broadly classified into three categories, i.e., replay, regularisation-based and parameter isolation [9]. **Replay** methods preserve some historical observation - either real or synthetic - to revisit when training on new tasks. **Regularization-based** methods use heuristics to determine the important weights in the neural network and to penalize changing these weights more when learning new tasks. Finally, **parameter isolation**, also referred to as architecture-based, reserves specific weights to be updated on specific tasks. This can be done by freezing part of the network, or extending the neural network for new tasks.

Specific evaluation metrics are used to evaluate continual learning methods. The most widely used are average accuracy, average forgetting and forward transfer [22, 1]. These evaluate the performance after learning all tasks. **Average accuracy** is the average of accuracy over the tasks, while average forgetting assesses the degradation of accuracy

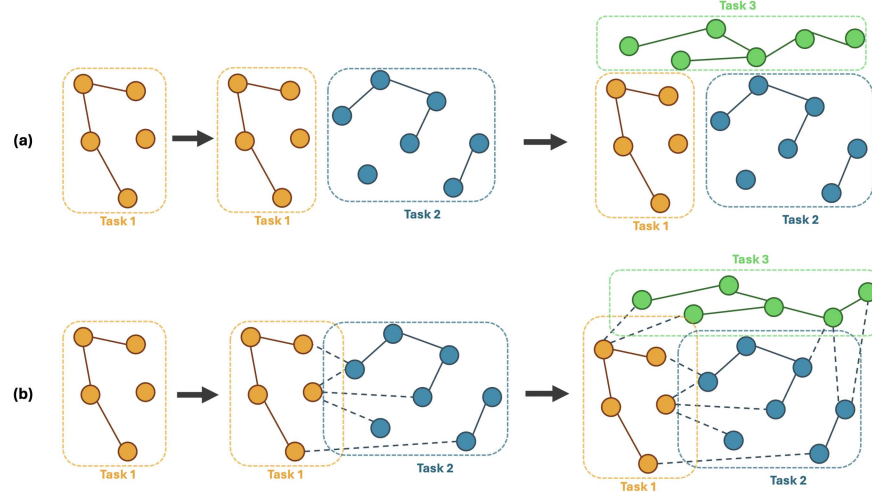


Figure 1: Two assumptions in continual graph learning for node-based learning. Either a separate network is provided for each task (a), or the network grows over time (b).

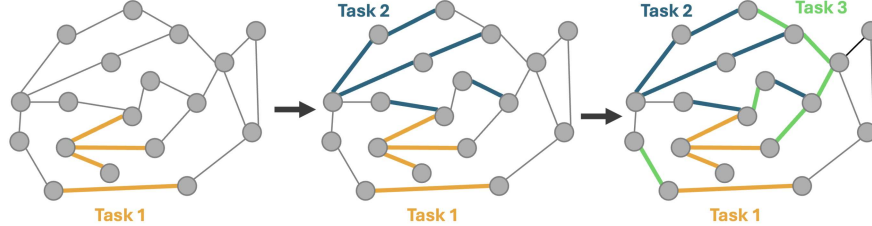


Figure 2: Visualisation of edge-based continual learning, where the network stays fixed, but the edge labels gradually become known.

over the tasks. **Average forgetting** compares the accuracy of a task after training on that tasks to the accuracy after learning on all tasks. Negative forgetting is sometimes also called backward transfer. **Forward transfer**, also called zero-shot learning, is the increase in accuracy when using the model trained on previous tasks, compared to random initialization [35, 1].

2.4 Continual Graph Learning

When using network data, additional considerations come into play for continual graph learning, since observations over different tasks may be connected in the network. As a result, information from previous tasks can still be leveraged when training the current tasks due to these inter-task connections. Suppose a classification problem, where each node is part of only one task [22]. The inter-task connections often fall within one of two categories, as visualised in Figure 1.

A first category is where each task consists of an independent network with no links to nodes in previous tasks [28, 22], so no inter-task connections are present. This can occur for two reasons. First, there are tasks which inherently do not have inter-task connections. This is often the case in graph classification [8], but also occurs when every sample in a task is a separate network. Second, separate tasks can also occur by design, where additional restrictions are put on the network by removing inter-task connections.

The second type of interaction is more prevalent, and involves a network that grows over time [61, 74, 14, 68, 73, 78]. With each new task, new nodes are added to the network. Some nodes in the new task have connections to nodes of previous tasks. Hence, special care is needed if we assume that not all data is available from previous tasks.

In this work, we also evaluate a continual learning setting where the network is fixed, but where the labels change over time, as shown in Figure 2. Here, the nodes are shared across tasks. This corresponds closely to real-life AML settings, where clients are monitored continuously. A banking client can start laundering money only after holding an account at the bank for a couple of years.

3 Literature Review

In the past years, deep representation learning has gained increased adoption for AML, although remaining under-explored [10]. In the same vein, the field of continual learning is mature, but less attention has been given to continual graph learning [73, 22]. To answer RQ1, this section provides an overview of that literature. An overview of the relevant literature on continual learning is given in Table 1

3.1 Graph Neural Networks for Anti-Money Laundering

Alarab et al. [2] present experiments with a GCN [20] to construct meaningful network embeddings for AML. Cases are classified by combining the embedding with original transaction features. Lo et al. [34] introduced inspection-L, which applies a GIN [67] and uses Deep graph infomax to find predictive node embeddings by comparing the embeddings of the real network to that of a corrupted network. The application of GNNs is extended by Jin et al. [19]. First, a heterogeneous interaction network is constructed to extract additional features. Then, the authors apply GCN [20], GraphSAGE [17] and GIN [67] on the homogeneous network to arrive at network embeddings augmented with the features from the heterogeneous network, so as to obtain the final predictions.

As money laundering typically occurs over a longer period of time, research has also aimed at extending GNNs to a spatio-temporal setting. This is often done by constructing embeddings on snapshots that are fed into a deep learning method for time series analysis. The work by [66] feeds the embeddings coming from a GCN [20] to an LSTM, while [72] applies a transformer model to the embedding vectors of the different snapshots.

Furthermore, GNNs have been used for AML anomaly detection. Cardoso et al. [7] use GAT [60] for link prediction between nodes in the network. These predictions are compared to the real links in the network, leading to anomaly scores that indicate suspicious transactions.

3.2 Continual (Graph) Learning Methods

When training (graph) neural networks in the classic way, it is assumed that the data is identically distributed, often with the possibility to shuffle it before using it to train the model [45]. However, data is typically not identically distributed in many real-life applications, where data becomes available in streams over time. An added difficulty arises when there is also a distributional shift in the tasks to learn. This can lead to catastrophic forgetting [15, 39, 16], where updating the model with new information leads to interference with earlier-acquired knowledge. The trade-off between the ability to retain old knowledge while also processing new information is called the stability-plasticity dilemma [9, 77, 62].

Continual learning, which was introduced to mitigate catastrophic forgetting, has mostly been applied for image classification [31]. One starts with a couple of images to classify, e.g., cat versus dog. The classes are then extended where the model also needs to be able to distinguish between, e.g., cars and planes. When learning this second task, the model should retain its ability to distinguish between cats and dogs.

Given the origin of the field, many of the proposed methods are also evaluated on image classification [48, 35, 21, 69, 29, 36, 9]. The most frequently used data sets are MNIST, CIFAR100 and ImageNet, and these also serve as the main data sets in benchmarking studies [9, 8, 58, 77].

Continual learning methods are classified in three categories, i.e., replay, regularisation-based and architecture-based [9, 31], whereas hybrid methods also exist.

Replay methods rely on a memory buffer for retaining a subset of data from previous tasks, called exemplars. These methods assume that resources are available to store previous data. Replay emerged in reinforcement learning [49], where past transitions are replayed for better learning on a single task, while replay in continual learning is used to retain information across tasks. Care should be taken when adopting such methods with regards to regulation. Due to privacy concerns, not all data is allowed to be stored indefinitely. Additionally, it is possible that these methods overfit on the few exemplars that are kept in memory [61].

Pure replay methods only specify a buffer size $\mathcal{B}$, and randomly assign $\mathcal{B}/k$ observations from the current task to be replayed during new tasks. This random selection is also applied by GEM [35] to select their exemplar set. iCarl [48], on the other hand, uses smart allocation by selecting exemplars that best preserve the average feature vector of that task.

Additionally, replay data can be generated synthetically. One way of doing this is proposed by Li and Hoiem [29] for their method Learning without Forgetting (LwF) in a task-IL setting. The authors assume that each task has its own output head. As data on previous tasks is not available, LwF takes the data of the current task, and makes predictions for the output heads of the previous tasks as well. This gives a new ‘ground truth’ for previous tasks, to which output is compared to during training, to keep the output on previous tasks stable.

Replay methods have also been extended to graph learning. Zhou and Cao [78] use the *mean of features* of selecting exemplars, and extend it for graph data by selecting exemplars based on the mean embedding, based on coverage maximisation and based on influence maximisation. Wang et al. [61] employ a two-step sampling approach where first the network is divided into clusters, after which nodes are selected within each cluster based on an importance metric.

For the class-IL setting, Liu et al. [33] proposes CaT, which first selects a subset of nodes randomly, and then uses a structure-free graph condensation method [76] to align the mean of latent features in the subset of nodes by training the input node features as weights.

Regularisation-based methods limit the updates to weights in the (graph) neural network. Determining which weights can change and by how much leads to a trade-off between the stability and plasticity of the model [45].

Regularisation-based methods often capture which weights are important for previous tasks, and limit how much these can be changed. This is done by introducing additional terms in the loss function. EWC [21] uses the elements in the Fisher information matrix to express which weights were important for previous tasks. As mentioned by Zenke et al. [69], EWC only makes point estimates of the importance using the diagonal elements of the Fisher information matrix. They put forward SI [69], where the importance of the weights are continuously calculated throughout the training process.

MAS [3] on the other hand, is based on the gradient of the squared l_2 -norm of the learning function output, making it applicable to unlabelled data as well. Simplifications of the importance calculations are given in case all layers use a ReLU activation function.

Regularisation can also be approached by altering the gradient before updating the weights. Using the exemplars in GEM, Lopez-Paz and Ranzato [35] project the gradient on the span of the gradients of the previous tasks. The authors claim that this does not increase the loss on older tasks when updating the weights for the new task.

Liu et al. [32] extends regularisation to network data with TWP. Two importance scores are included, i.e., a task-related one similar to EWC and a topology-related one based on the attention mechanism in graph attention networks (GAT) [60]. Similarly, to mitigate the problem of overfitting on the exemplars, Wang et al. [61] applies the same idea of EWC to GNNs. The authors use the diagonal elements of the Fisher information matrix to regularize the updates of important weights in the GNN.

Architecture-based methods, as their name suggests, alter the architecture of the (graph) neural network based on the tasks. Specific parameters can be isolated to be fine-tuned, or the architecture itself can be extended for new classes, e.g., have separate output heads for each task [29]. Here, knowledge of the current task must be provided. Some methods also assume that the total number of tasks is known upfront. This limits their adoption in practice.

van de Ven et al. [58] mention the use of entirely separate output layers or networks to learn each task. In the task-IL setting, Li and Hoiem [29] initialize a new output layer for each new task. Other methods include the usage of gating. XdG [38] introduced a context-dependent gating signal, to have sparsely connecting, mostly non-overlapping parts of the network trained on the different tasks.

Similar to gating, PackNet [36] and piggyback [37] apply task-specific masks to set some weights in the neural network equal to 0. When learning a task, Mallya and Lazebnik [36] train the network and then prune it. The remaining weights are then fine-tuned in a second, shorter training round. These weights are fixed and the pruned weights are made available for the next task. The main drawback is that less capacity is available for training on the next task, meaning that PackNet can only learn a limited number of tasks. This is mitigated in the work by Mallya et al. [37]. Here, the network weights are fixed and the task-specific masks are learned. The authors use gradient-based learning to obtain real-valued masks, which are then converted to binary masks using a fixed threshold. The main drawback here, next to the need to know which task is considered, is that piggyback requires a pre-trained network. The performance of piggyback is highly dependent on this pre-training step [37], and this might even not be available depending on the application.

3.3 Continual Learning in Financial Fraud

When implementing fraud detection in practice, the methods need to continuously monitor millions of transactions, which results in three main challenges. The first is that, given that fraud is evolving constantly [5, 59], AML models should be updated to capture novel *modi operandi*. The second is constraints on computational resources. The large volume of transactions make retraining the model from scratch not always feasible given limited time and resources. The third challenge comes from the desire to retain knowledge on previously applied fraud tactics, because launderers could otherwise revert back to an older *modus operandi* to avoid detection. However, when fine-tuning the

Paper	Graph Data	Fraud	GCN	GraphSAGE	GAT	GIN	Replay	iCarl	EWC	MAS	GEM	TWP	LwF	ER-GNN	PackNet	Piggyback	HAT	CuT	PI-GNN	CGNN
Zhang et al. [73]	✓	×	✓	×	×	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Ko et al. [22]	✓	×	✓	×	×	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Zhou et al. [77]	✓	×	✓	×	×	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
De Lange et al. [9]	✓	×	✓	×	×	✓	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Febrinanto et al. [14]	✓	×	✓	✓	✓	✓	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Zhang et al. [74]	✓	×	✓	×	×	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Wei et al. [65]	✓	×	✓	×	×	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Wang et al. [61]	✓	✓	✓	✓	×	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Hernani et al. [18]	✓	✓	✓	×	✓	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Zhang et al. [71]	✓	✓	✓	×	✓	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Li et al. [28]	✓	✓	✓	×	✓	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
van de Ven et al. [58]	✓	✓	✓	×	✓	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Carta et al. [8]	✓	✓	✓	×	✓	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Zhou and Cao [78]	✓	✓	✓	×	✓	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Lebichot et al. [23]	✓	✓	✓	×	✓	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Perini et al. [46]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Zhang et al. [71]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Table 1: Table summarising the main literature in continual (graph) learning.

models, this information is lost if these tactics were not applied in the latest available data, resulting in catastrophic forgetting [15, 39, 16].

Although continual learning can be used to mitigate these challenges, research on the adoption of continual learning for fraud detection is scarce [31]. Research on continual learning is mostly concerned with image recognition [31]. Few studies present fraud detection as the core problem [18, 71, 23, 28]. It often only appears as one of many data sets to which continual learning methods are applied [61, 46, 22].

Lebichot et al. [23] were among the first to quantify catastrophic forgetting for credit card fraud detection. They tested different replay strategies and EWC against fine-tuning the model. In their case, fine-tuning the model on new data seemed to be best at avoiding forgetting.

Hemati et al. [18] applied three strategies, sequential fine-tuning, EWC, and experience replay, for auditing financial payment records. They trained auto-encoders for anomaly detection and demonstrated that continual learning has the ability to detect distributional shifts.

Zhang et al. [71] introduced and applied a new method, called POCL, to medical insurance fraud detection. The authors rely on Temporal MAS to update the weights of their GNN.

Li et al. [28] extended continual graph learning to a case study on heterogeneous networks by introducing HTG-CFD. They apply replay and regularisation-based methods, where prototypes are constructed using the average attribute, and regularisation is done using Fisher information, inspired by EWC. HTG-CFD is constructed to transfer the fraud detection model across different regions to detect fraudulent transactions in a trade network.

ContinualGNN [61] is developed for streaming graphs to uncover new patterns over time. The authors have tested their method on the Elliptic data set. Although ContinualGNN did not perform best, the method has competitive performance. The main strength of this method is the strong reduction in training time compared to fully retraining the GNN with new data.

3.4 Benchmarks and Evaluation in Continual (Graph) Learning

De Lange et al. [9] are among the first to do an extensive benchmark study. While focusing only on task-incremental learning in classic continual learning, they implement a comprehensive benchmark both in terms of methods as well as data sets. These data sets all involve image classification. They conclude that architecture-based methods, particularly PackNet [36], perform best, closely followed by memory replay. However, compared to memory replay, architecture-based methods do not suffer from privacy issues, since they do not require storing raw data.

One of the first benchmark studies for continual graph learning was presented by Carta et al. [8]. This study only considers graph classification, so no tests are done at node level. It is presented as an introductory benchmark experiment and it is quite limited in its scope. It involves three data sets and three continual learning strategies. These are naive replay, EWC [21] and LwF [29]. Hence, although the paper is meant to be a benchmark on networks, no strategies that were specifically developed for networks were tested.

Two other notable benchmark studies for continual graph learning are *Continual Graph Learning Benchmark (CGLB)* by Zhang et al. [73] and *Benchmarking Graph Continual Learning (BeGin)* by Ko et al. [22], both of which provide the full code suite to facilitate reproduction. The initial methods compared by Zhang et al. [73] are EWC [21], MAS [3], GEM [35], TWP [32], LwF [29] and ER-GNN [78]. CGLB split continual graph learning in task-IL and class-IL and provide experiments for node-level and graph-level predictions.

A more extensive benchmark is implemented by BeGin [22]. They make a fine-grained distinction between incremental settings, by considering task-IL, class-IL, domain-IL and time-IL. These settings are also introduced for link-level predictions, on top of the earlier introduced node-level and graph-level predictions. The continual learning methods are also extended. On top of the methods compared under CGLB, BeGin also includes PackNet [36], Piggyback [37], HAT [52], CaT [33], PI-GNN [70] and CGNN [61].

Both benchmarks analyse the performance of a range of methods, but pay less attention to the impact of the hyperparameters. One of the main shortcomings of both CGLB and BeGin is that all experiments are done only with GCN [20] as backbone GNN.

3.5 Sensitivity to Hyperparameters

Underlying every model is a suite of hyperparameter choices that impact model performance. These are often only briefly mentioned under hyperparameter tuning, or in the best case, papers apply limited parameter sensitivity tests. In this work, we make the effect of hyperparameters explicit.

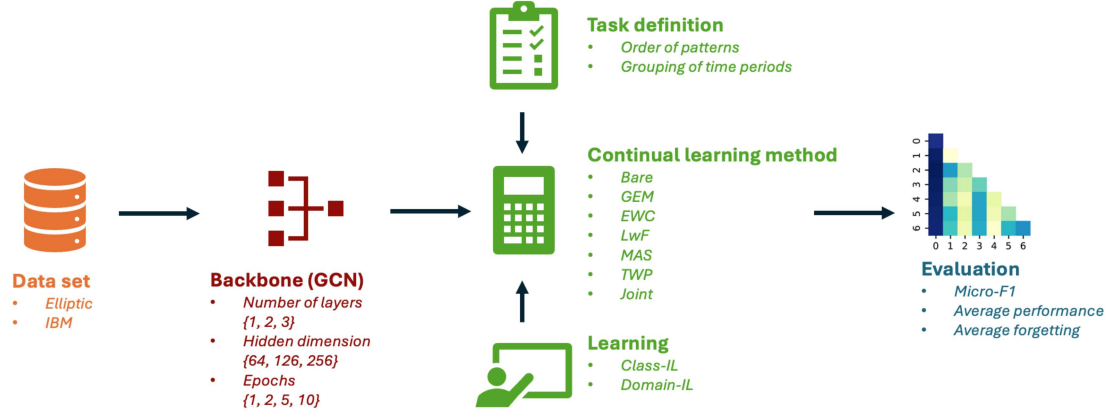


Figure 3: Visualisation of the experiment’s pipeline and the hyperparameter choices for evaluation.

A key choice is the backbone GNN model. In the literature, GCN [20] and GAT [60] are popular backbones, as indicated by Yuan et al. [68] and corroborated by our observation in Table 1. Most continual graph learning methods are constructed to be backbone agnostic. One notable exception is TWP [32], which is specifically developed with GAT in mind. However, the authors provide proxies in case no attention mechanism is present.

The architecture of the backbone GNN itself - both in terms of depth and width of the hidden layers - is also important since it influences the learning capacity and the length of transaction chains that can be captured. However, the specific impact of these choices have not been given much attention in literature. Studies on general continual learning by De Lange et al. [9] and Mirzadeh et al. [40] demonstrated that wide and shallow models generally perform better. However, when moving to continual graph learning, Wei et al. [64] demonstrated that for skeleton-based action recognition this does not always hold.

Another import choice, next to the backbone, is the task definition. When considering human learning, it is hypothesized that the order of tasks is important for continual learning. *Curriculum Learning*, coined by Bengio et al. [6], determines that knowledge can be optimally acquired if tasks are learned in ascending order of difficulty.

Previous work has performed task-order sensitivity analysis in continual learning. De Lange et al. [9] corroborated earlier work of Nguyen et al. [43] by showing that, in a general continual learning setting, methods exhibit order-agnostic behaviour. The authors test different setups, including an *easy to hard*, a *hard to easy*, and a random ordering of tasks.

The work by Mallya and Lazebnik [36], on the other hand, showed that for their method PackNet, the order does matter. They found that learning tasks from hardest to easiest actually gave better results. We cannot generalise this finding, however, since this is method-specific. The capacity of PackNet to incorporate novel information drops as the available free parameters decrease with each task.

In the field of continual graph learning, Zhao et al. [75] introduced a randomly generated class appearance order to simulate the random class emergence in real world for multi-label continual graph learning. Wei et al. [64] focused on evaluating the task-order and class-order sensitivity in the context of continual graph learning for skeleton-based action recognition. The authors show that task-order robustness does not necessarily imply class-order robustness.

4 Methodology

To answer the research questions, we set up a pipeline in which we vary the different hyperparameters (RQ2), the architecture of the GNN (RQ3) and the continual learning methods (RQ4) to see their effect on performance and forgetting. Figure 3 illustrates the full pipeline of our experiments, including the value of the hyperparameters.

The experiments presented in this paper are an extension of the BeGIn framework that was introduced by Ko et al. [22]. The extended repository is made available on github³.

³<https://github.com/VerbekeLab>

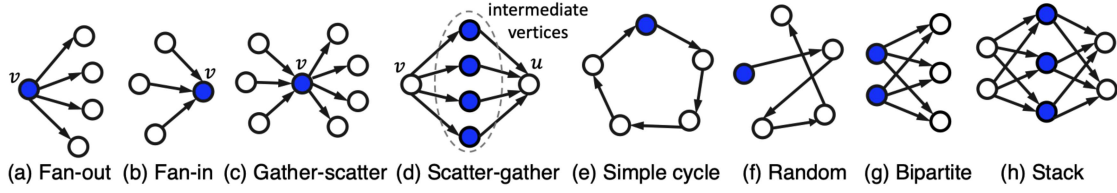


Figure 4: The different money laundering patterns as defined by Altman et al. [4].

Type of pattern	Transactions
Fan-out	342
Fan-in	318
Gather-scatter	716
Scatter-Gather	626
Cycle	287
Random	191
Bipartite	263
Stack	466
Not classified	1 968
Total money laundering	5 177
Total number of transactions	5 078 345

Table 2: The distribution of the different types of money laundering transaction patterns for the HI-Small data sets.

4.1 Data

Two data sets are used in the experiments that are widely used in AML [10], i.e., the IBM AML data set [4] and the Elliptic data set [13, 63]. As will be discussed below, this work presents AML both as an edge classification (IBM) as well as a node classification (elliptic) problem.

The IBM AML data set [4] contains synthetic transaction data. The simulations are done via a virtual multi-agent virtual world. These agents can be banks, individuals or companies, with payments by individuals and companies. In this virtual world, some agents are said to be malicious. For those agents, the simulations include money laundering transactions. Altman et al. [4] model eight different money laundering patterns, i.e., fan-in, fan-out, bipartite, stack, random, cycle, scatter-gather and gather-scatter, as illustrated in Figure 4.

We select the HI-Small data set to use for our evaluations, taking the agents as nodes and the transactions as edges. This results in a network with 515 080 nodes and 5 078 345 edges. For this data set, we perform edge classification. As with most fraud data sets, the class distribution is highly imbalanced, with only 0.1% of transactions involving money laundering. Most of the money laundering transactions are, however, not classified under a specific pattern. Table 2 gives a detailed view on the distribution of the class labels. We will discard the *not classified* labels in our experiments, since we have no control over the specific patterns they constitute. This is also a practice observed in other research when the specific patterns are classified [4, 12].

The Elliptic data set [13, 63] contains real-world Bitcoin transactions, grouped in 49 different time intervals. The network consists of 20 3769 nodes and 234 355 edges, where nodes represent transactions and edges indicate that the receiver of the first transaction was the sender of the second. For this data set, we perform node classification. The data set includes 166 pre-calculated numerical features — 94 transaction-specific features and 72 aggregated features summarizing a node’s neighbours. The data contains only 4 545 illicit transactions (2%), again making the label distribution highly imbalanced. Although these labels do not specifically concern money laundering, we use this data set as it has found wide adoption in the AML literature [63, 10, 2, 66, 41, 53, 26, 30], and therefore facilitates comparison with prior experimental results.

Additionally, the Elliptic data set provides a well-suited case-study for continual learning [46, 61]. As mentioned by Weber et al. [63], there was a sudden closure of a dark market at time step 43. This caused all methods to perform poorly, due to the sudden shift in feature distribution of the illicit cases. This abrupt change in a real-world data set is ideal for getting a deeper understanding on continual learning methods.

4.2 Backbone Graph Neural Network

In line with previous benchmarks [73, 22], we use the GCN [20] as backbone. The GCN layer-wise propagation is defined for the whole network at once as:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right), \quad (2)$$

with σ an activation function, and $W^{(l)}$ the layer-specific trainable weights. We limit this study to GCN, as this is currently the only backbone implemented in BeGin [22]. Other popular backbones could be GraphSAGE and GAT, but a far-reaching extension of BeGin is outside the scope of this work. Additionally, GAT has many more parameters to learn, which would substantially increase calculation time.

The main choices for the backbone are the depth and width of the layers [9, 40, 64]. The number of layers varies between 1 and 3. Here, a trade-off needs to be made. On the one hand, money laundering patterns often contain nodes that are a couple of hops in the network apart, requiring more layers to capture this information. On the other hand, having too many layers in a GNN leads to over-smoothing [27], lowering the predictive power of the model. The dimensions are the same for all hidden layers in the GCN. We test three values, namely 64, 128 and 256.

Another choice concerns the number of epochs per task, which we set to 1, 2, 5 and 10. The more time a model is given to train on a single task, the more likely the weights are changed and previous knowledge is lost. On the other hand, the fewer epochs used, the harder it is for the model to learn and improve performance on the current task.

Some hyperparameters of the GCN are fixed. For all experiments, we use the Adam optimizer with learning rate 0.001 and no weight decay and cross-entropy loss. The activation function for the GCN is ReLU, and the dropout rate is set to 0.5.

4.3 Task Definition

The elliptic data set contains fraud/non-fraud labels, and we therefore apply binary classification. We can take each time step as an individual task, resulting in 49 tasks. As this many tasks might pose a problem for the continual learning methods, we also group different time steps per task. Hence, we opt to also have seven time steps per task. The latter is chosen to have the same number of time steps in each task.

The patterns in the IBM data set each have their own separate label, resulting in multi-class classification. For the IBM data set, the tasks are defined using the different patterns present in the data set. The first task will consist of two labels, i.e., legitimate transactions and a first money laundering pattern. Subsequent tasks are defined by adding one novel pattern at a time. We analyse the sensitivity to the order in which the tasks are presented. The different ways represent different scenarios that can occur when applying continual learning for AML in practice.

A first way is to present the patterns in ascending order of difficulty. When a bank starts implementing AML procedures, investigators are not yet experts in all patterns. They start by mastering simpler patterns, and gradually become more familiar with more complex money laundering patterns. This is also a result of the cat-and-mouse game between institutions and criminals. If financial institutions become better at detecting specific patterns, launderers will adapt and resort to more complex operations, prompting institutions to learn to detect these more complex patterns. To see if the complexity of the pattern plays a role, we also present the patterns in decreasing order of difficulty.

The determination of what patterns are more difficult than others is inspired by the work of Egressy et al. [12]. The authors extend message-passing GNNs step-by-step to prove that the extended GNNs can capture more patterns. We use those insights to order the patterns from least to most complex as follows: fan-in, fan-out, bipartite, gather-scatter, scatter-gather, stack, cycle and random.

A second way to present the patterns is in descending order of frequency, i.e., gather-scatter, scatter-gather, stack, fan-out, fan-in, cycle, bipartite, random. It is hypothesized that more frequent patterns will be noticed sooner. This also makes the subsequent tasks more difficult to learn, since each time there are fewer observations to learn from. On the other hand, the model might be able to achieve better performance on these new tasks by transferring knowledge from previous tasks. In addition, we will also present the task in reverse order, from least to most frequent.

Finally, as a baseline approach, we also present the patterns in random order, which for our experiments is fan-out, fan-in, gather-scatter, scatter-gather, cycle, random, bipartite and stack.

Note that for the IBM data set, the transaction network is static. We incrementally learn the novel patterns, while keeping all nodes and edges the same.

4.4 Continual Learning Methods

Different methods to prevent forgetting are present in continual graph learning literature, starting with the type of incremental learning to use. For the IBM data set, we use class incremental learning, since each new pattern is seen as its own class. The main problem as indicated in the literature is that these different tasks in the class-incremental setting are very similar, possibly resulting in strong forgetting across tasks [62, 24].

Domain-IL is used for the elliptic data set, where we recognise that the distribution in money laundering patterns can change. This mimics what happens in reality, since financial institutions will also use binary classification (i.e., legit vs. money laundering), but need to consider that the modus operandi of fraudsters evolves over time.

The continual learning methods used are Gradient Episodic Memory (GEM) [35], Elastic Weight Consolidation (EWC) [21], Learning without forgetting (LWF) [29], Memory Aware synapses (MAS) [3], and Topology-aware Weight Preserving (TWP) [32]. This selection is made since the literature as presented in Section 3.3 also relies on these methods for fraud detection. These methods are compared to the bare and joint model.

- **Bare:** We iteratively fine-tune the model on only the data of the current task. In continual learning, this is taken as a lower bound for the performance.
- **Gradient Episodic Memory (GEM) [35]:** GEM uses a fixed budget for memory allocation, and this memory is filled without any smart allocation of replay. The gradient of the current task is projected onto the space spanned by the gradients calculated using the replay examples, to avoid that the losses on previous tasks will increase.
- **Elastic Weight Consolidation (EWC) [21]:** EWC uses the Fisher information matrix, based on the gradient of the loss, to find the weights that were important for the previous task. It changes the loss function by introducing heavier penalization of updating more important weights.
- **Learning without forgetting (LWF) [29]:** LwF is introduced using a multi-task architecture, where part of the model is shared, and part is fine-tuned for each specific task. It assumes that no data of older tasks is available. LwF starts by constructing new ‘ground truth’ labels by looking at the output on the parts of the old task using the data of the current task. Original capabilities are preserved by trying to keep these outputs as is while training on the new task.
- **Memory Aware synapses (MAS) [3]:** MAS uses the gradient of the squared l_2 -norm of the learned function output to express weight importance. Contrary to EWC, the weight importance determined by MAS is done in an unsupervised manner.
- **Topology-aware Weight Preserving (TWP) [32]:** TWP uses two sub-modules, one for task-related objectives and one for topology-related objectives. The task-related objective is similar to EWC where weight importance is measured via the gradient of the loss. The topology-related objective relies on the gradient vector of the attention coefficients in a GAT to incorporate network topology. The authors include a non-parametric proxy for the attention, in case the GNN backbone does not include an attention mechanism.
- **Joint:** the joint model takes an accumulative approach. Similar to the bare model, the model of the previous task is fine-tuned on the current task. Contrary to the bare model, the joint model is fine-tuned using all data of the current and past tasks. In continual learning, this is taken as an upper bound for the performance.

4.5 Evaluation

We focus on average forgetting and average performance. Previous work mostly uses accuracy to evaluate performance [1]. However, AML deals with strong class label imbalance, motivating the evaluation of performance by using the micro-F1 score for both data sets.

In continual learning, special evaluation metrics are developed to reflect that the model is fine-tuned sequentially on the tasks. We use the model tuned for task j , and evaluate it on the test data of previous tasks $i \leq j$. This allows us to quantify the forgetting that has occurred after fine-tuning the model on task j .

Using this principle, we define the performance matrix [73], $M \in \mathcal{R}^{k \times k}$, with k the number of tasks. The elements of the performance matrix are defined as:

$$M_{i,j} = \begin{cases} \text{Performance on task } i \text{ after training on task } j & \text{if } i \leq j \\ 0 & \text{otherwise} \end{cases}$$

Hence, the performance matrix is a lower triangular matrix. The visualisation of the performance matrix using a heatmap is a first, qualitative evaluation of the methods.

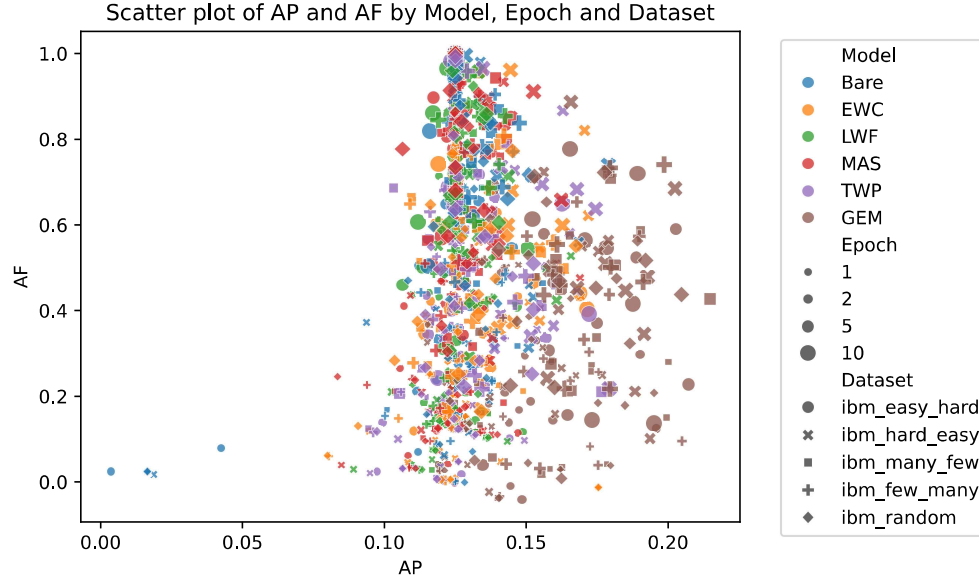


Figure 5: Scatterplot of the average forgetting plotted against the average performance for the IBM data set.

The performance matrix entries are used to calculate quantitative evaluation metrics, i.e., the average performance (AP) and average forgetting (AF). Average performance is the average micro-F1 over the tasks, after training on all tasks. Average forgetting compares the micro-F1 of a task after training on said tasks to the accuracy after learning on all tasks. Using the performance matrix, we defined these metrics as [22]:

$$AP = \sum_{i=1}^k \frac{M_{k,i}}{k} \quad (3)$$

$$AF = \sum_{i=1}^{k-1} \frac{M_{i,i} - M_{k,i}}{k-1} \quad (4)$$

In the end, we are also interested in the performance of the final model on all tasks. Therefore, we include the micro-F1 score on all data after fine-tuning the model on the final task.

5 Results and Discussion

As shown in the pipeline of Figure 3, many hyperparameters choices are tested in this work. We start below with a general overview of all results in which some trends are already clear. Afterwards, a detailed discussion for each choice is given.

We provide the full set of results on the average forgetting and average performance in the figures below. Figure 5 contains a scatter plot of the results over all different (hyper-)parameters for the IBM data set. We see strong forgetting across experiments, which is probably caused by the strong similarity among the different tasks [62].

We notice that in general more epochs lead to more forgetting, while there is a limit on the performance the model can obtain. Additionally, GEM seems to achieve good performance without suffering too much forgetting.

A similar figure is given for the Elliptic data set. Figure 6 has different results. For the Elliptic data set, it seems that forgetting is less of a problem, but the average performance clearly increases with the number of epochs. In general, the results are similar between the setting with seven and the one with 49 tasks. The lack of forgetting can be because the distribution shift less severe in this data set.

Looking at the methods, it seems that also here GEM has on average lowest forgetting, while LwF and Bare have higher forgetting. The difference in forgetting among methods for the Elliptic data set is, however, much less pronounced than for the IBM data set.

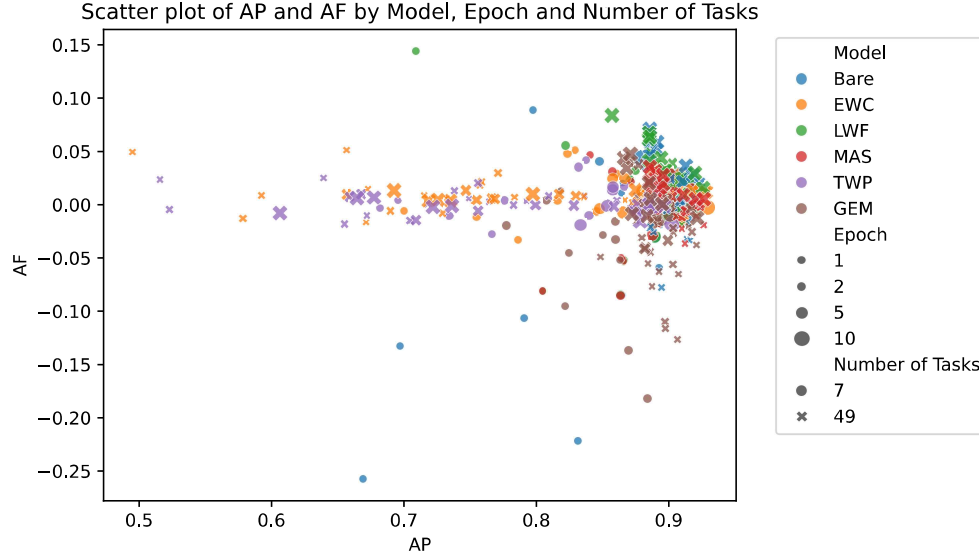


Figure 6: Scatterplot of the average forgetting plotted against the average performance for the Elliptic data set.

A more fine-grained visualisation for the IBM data set is provided in Figure 7. The results are split in different plots according to the depth and width of the model. We see that higher dimensions lead to an increase in performance, but also correlate with stronger forgetting. We also note that the Bare model suffers strong forgetting, especially for the GCNs with three layers. For the other layers, Bare, MAS and LwF seem to consistently suffer more from forgetting for similar performance, compared to the other methods.

5.1 Number of Epochs

To illustrate the importance of carefully choosing the number of epochs, the evolution of the average forgetting as a function of the number of epochs is shown in Figure 8 for the IBM data set. We can clearly see that the average forgetting stays relatively low for a couple of epochs, but suddenly jumps up. This illustrates that, in this case, knowledge from previous tasks is not lost gradually, but suddenly.

We note that the average forgetting drops when going to a new task, after every five epochs. This is because a new task is added to the average forgetting calculations. As the model has just fine-tuned on data from that task, the forgetting is expected to still be low. This results in a lower average, and hence a drop in forgetting when considering the next task.

The average forgetting, average performance and final performance are reported for the IBM data set in Table 3 over the different number of epochs per task. We see that performance improves when increasing the number of epochs. However, the amount of forgetting becomes a major issue with a higher number of epochs. It seems that only GEM is able to keep forgetting at a lower level.

We see that the final performance goes down considerably when the number of epochs per task goes up. This is caused by the high forgetting for these cases. Especially forgetting for the first task is problematic, given that it contains the majority class.

The average forgetting, average performance and final performance on all data are reported for the Elliptic data set in Table 4 for seven tasks and in Table 5 for 49 tasks over the different number of epochs per task. As before, the performance increases with the number of epochs per task, while forgetting increases, but not as drastically as for the IBM data set. Here, both TWP and GEM seem to be able to achieve high performance in combination with negative forgetting, meaning that the model also improves its performance on previous tasks when fine-tuning on novel tasks.

The final performance of the models seems less affected by the number of epochs. This is probably caused by the fact that there is very little forgetting since there is limited data shift over the tasks. Therefore, the model has had ample training time at the end of fine-tuning on all tasks, even when the number of epochs per task is small.

Something we notice for the IBM data set in Figure 8 and Table 3 is that the Bare model, although considered as a lower bound, does not suffer the strongest forgetting of all models. There can be a couple of reasons for this. A first reason is that the network structure, via the inter-task connections, is beneficial for the bare model to retain knowledge

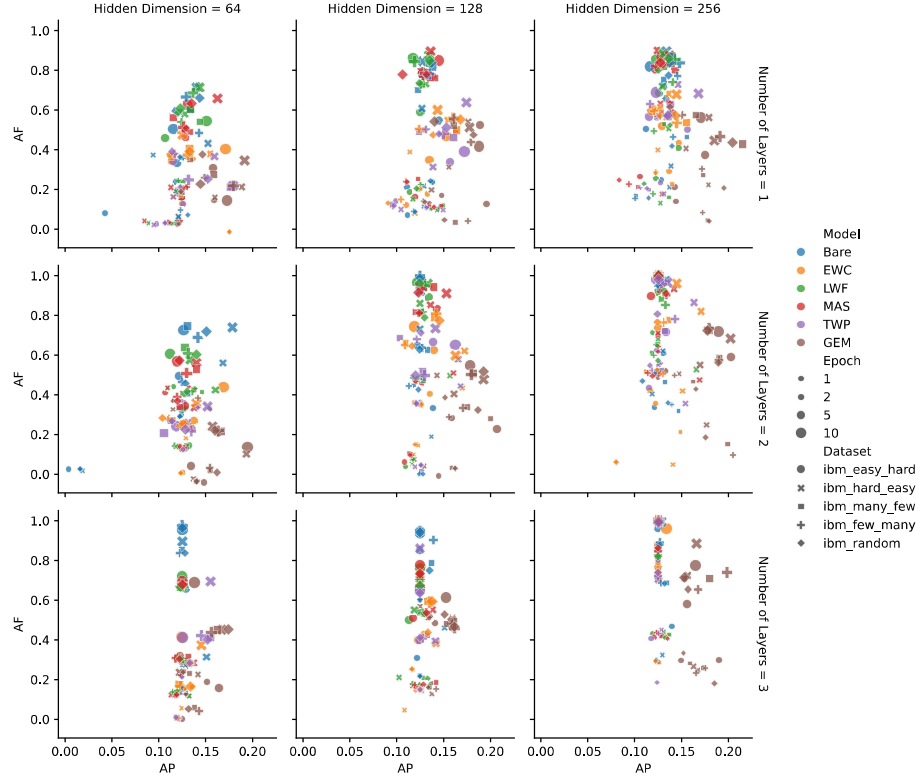


Figure 7: Scatter plots of the average forgetting (lower is better) plotted against the average performance (higher is better), split according to the breadth and width of the GCN.

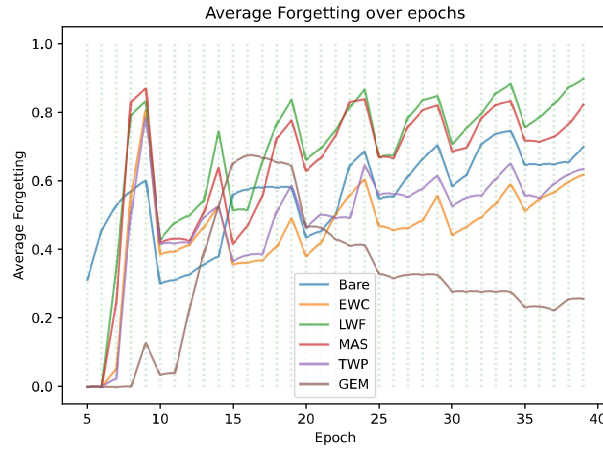


Figure 8: Average forgetting for the different methods on the IBM data set, where the backbone is a GCN with two layers, each having dimension 128, and we take five epochs per task.

Epochs	Bare	EWC	LWF	MAS	TWP	GEM
1	AP: 0.1250 AF: 0.0000 Fin.: 0.0004	AP: 0.1353 AF: 0.4036 Fin.: 0.0003	AP: 0.1095 AF: 0.0394 Fin.: 0.7800	AP: 0.1084 AF: 0.0606 Fin.: 0.7431	AP: 0.1272 AF: 0.0171 Fin.: 0.6529	AP: 0.1444 AF: -0.0100 Fin.: 0.6372
2	AP: 0.1385 AF: 0.3340 Fin.: 0.0001	AP: 0.1259 AF: 0.3501 Fin.: 0.0076	AP: 0.1253 AF: 0.4897 Fin.: 0.0001	AP: 0.1265 AF: 0.4569 Fin.: 0.0001	AP: 0.1250 AF: 0.3574 Fin.: 0.0002	AP: 0.1588 AF: 0.2823 Fin.: 0.0339
5	AP: 0.1220 AF: 0.6489 Fin.: 0.0001	AP: 0.1393 AF: 0.6274 Fin.: 0.0002	AP: 0.1336 AF: 0.8924 Fin.: 0.0001	AP: 0.1425 AF: 0.8318 Fin.: 0.0001	AP: 0.1389 AF: 0.6642 Fin.: 0.0002	AP: 0.2072 AF: 0.2277 Fin.: 0.2783
10	AP: 0.1250 AF: 0.9804 Fin.: 0.0001	AP: 0.1192 AF: 0.7433 Fin.: 0.0001	AP: 0.1222 AF: 0.9657 Fin.: 0.0001	AP: 0.1250 AF: 0.9576 Fin.: 0.0001	AP: 0.1627 AF: 0.6503 Fin.: 0.0003	AP: 0.1783 AF: 0.5456 Fin.: 0.2337

Table 3: Performance metrics (AP and AF) and final performance on all data (Fin.) for different models and epochs for the IBM data set. The backbone is a GCN with two layers, each having dimension 128.

Epochs	Bare	EWC	LWF	MAS	TWP	GEM
1	AP: 0.8318 AF: -0.2213 Fin.: 0.8284	AP: 0.8903 AF: -0.0010 Fin.: 0.9019	AP: 0.8908 AF: 0.0012 Fin.: 0.9029	AP: 0.8922 AF: 0.0000 Fin.: 0.9041	AP: 0.8900 AF: -0.0018 Fin.: 0.9020	AP: 0.8860 AF: -0.0196 Fin.: 0.8930
2	AP: 0.8971 AF: -0.0012 Fin.: 0.9074	AP: 0.8936 AF: -0.0035 Fin.: 0.9028	AP: 0.8946 AF: 0.0012 Fin.: 0.9057	AP: 0.8973 AF: -0.0026 Fin.: 0.9077	AP: 0.8936 AF: -0.0011 Fin.: 0.9050	AP: 0.8696 AF: -0.1368 Fin.: 0.8771
5	AP: 0.9004 AF: 0.0156 Fin.: 0.9102	AP: 0.9037 AF: 0.0045 Fin.: 0.9113	AP: 0.9040 AF: 0.0158 Fin.: 0.9133	AP: 0.8978 AF: 0.0166 Fin.: 0.9085	AP: 0.9063 AF: -0.0093 Fin.: 0.9150	AP: 0.9097 AF: -0.0107 Fin.: 0.9168
10	AP: 0.9084 AF: 0.0287 Fin.: 0.9149	AP: 0.9156 AF: 0.0116 Fin.: 0.9204	AP: 0.9091 AF: 0.0248 Fin.: 0.9157	AP: 0.8996 AF: 0.0275 Fin.: 0.9099	AP: 0.9162 AF: -0.0026 Fin.: 0.9229	AP: 0.9175 AF: -0.0023 Fin.: 0.9227

Table 4: Performance metrics (AP and AF) and final performance on all data (Fin.) for different models and epochs for the Elliptic data set, with seven tasks. The backbone is a GCN with two layers, each having dimension 128.

Epochs	Bare	EWC	LWF	MAS	TWP	GEM
1	AP: 0.9146 AF: -0.0173 Fin.: 0.9229	AP: 0.7328 AF: 0.0060 Fin.: 0.7200	AP: 0.8978 AF: 0.0105 Fin.: 0.9100	AP: 0.9129 AF: -0.0081 Fin.: 0.9213	AP: 0.7927 AF: 0.0033 Fin.: 0.7821	AP: 0.8756 AF: -0.0155 Fin.: 0.8778
2	AP: 0.9144 AF: 0.0023 Fin.: 0.9236	AP: 0.7584 AF: 0.0215 Fin.: 0.7431	AP: 0.9157 AF: -0.0014 Fin.: 0.9250	AP: 0.9203 AF: -0.0107 Fin.: 0.9269	AP: 0.7557 AF: 0.0204 Fin.: 0.7472	AP: 0.8798 AF: -0.0316 Fin.: 0.8824
5	AP: 0.8870 AF: 0.0507 Fin.: 0.9031	AP: 0.8764 AF: 0.0024 Fin.: 0.8707	AP: 0.8913 AF: 0.0372 Fin.: 0.9058	AP: 0.9133 AF: -0.0058 Fin.: 0.9225	AP: 0.8766 AF: 0.0005 Fin.: 0.8860	AP: 0.9168 AF: -0.0035 Fin.: 0.9211
10	AP: 0.8913 AF: 0.0583 Fin.: 0.9050	AP: 0.7977 AF: 0.0100 Fin.: 0.7794	AP: 0.8849 AF: 0.0617 Fin.: 0.9020	AP: 0.8897 AF: 0.0316 Fin.: 0.9044	AP: 0.7362 AF: -0.0010 Fin.: 0.7182	AP: 0.8947 AF: 0.0009 Fin.: 0.8961

Table 5: Performance metrics (AP and AF) and final performance on all data (Fin.) for different models and epochs for the Elliptic data set, with 49 tasks. The backbone is a GCN with two layers, each having dimension 128.

from previous tasks. This reason seems to be supported by the results of the Elliptic data set in Table 4, where the Bare model seems to be performing worse, although not in all cases. Inter-task connections are absent in the Elliptic data set. A second reason for the deviant performance of the Bare model on the IBM data set is visible in the performance matrices in Figure 9. It seems that for some tasks, the bare model has difficulty obtaining good performance, leading to less *learned information* to forget.

Further analysis of the performance matrices in Figures 9-11 illustrate that a balance needs to be struck between performance and forgetting. The forgetting is kept low with a lower number of epochs, although the model does not

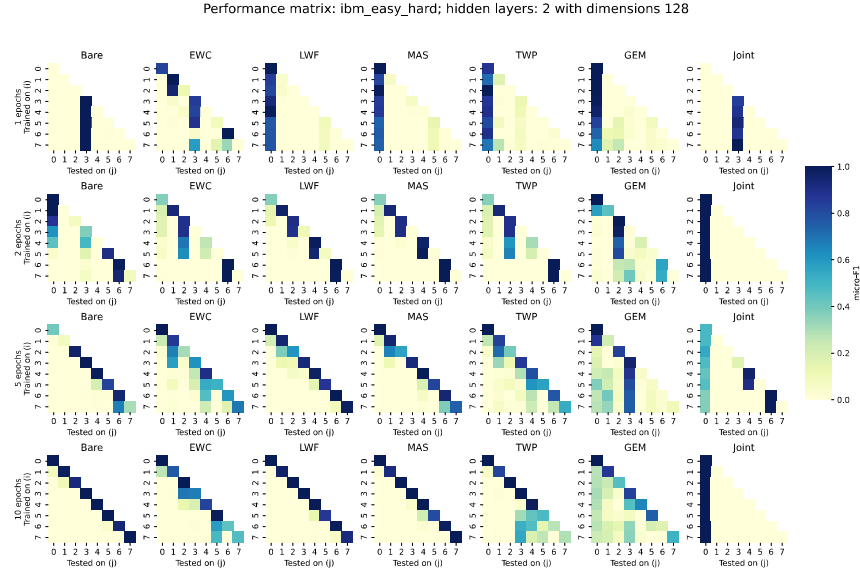


Figure 9: The performance matrices for the different methods for a varying number of epochs for the IBM data set.

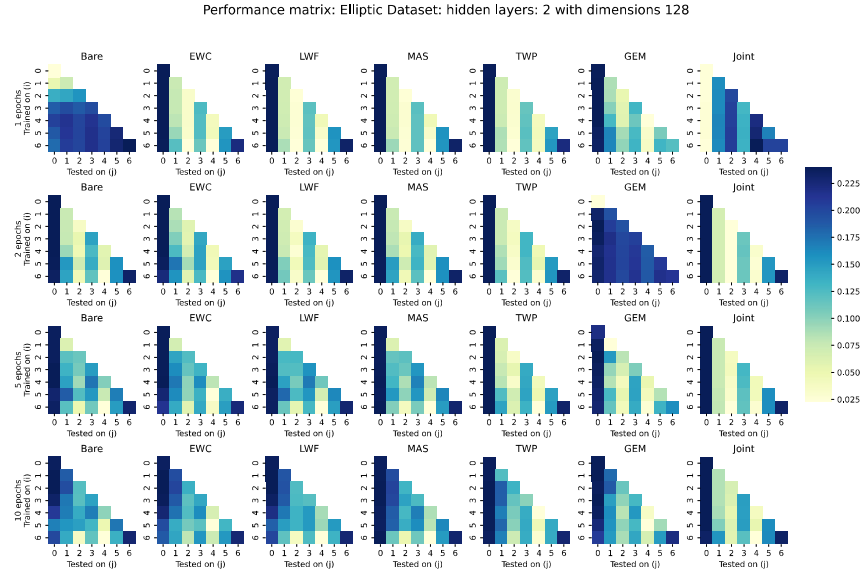


Figure 10: The performance matrices for the different methods for a varying number of epochs for the Elliptic data set with 7 time steps per task.

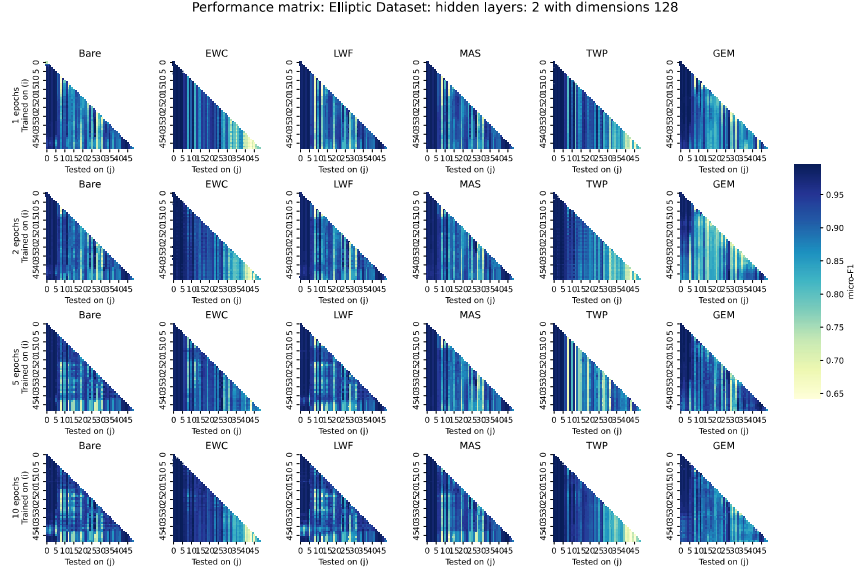


Figure 11: The performance matrices for the different methods for a varying number of epochs for the Elliptic data set with each time step a separate task.

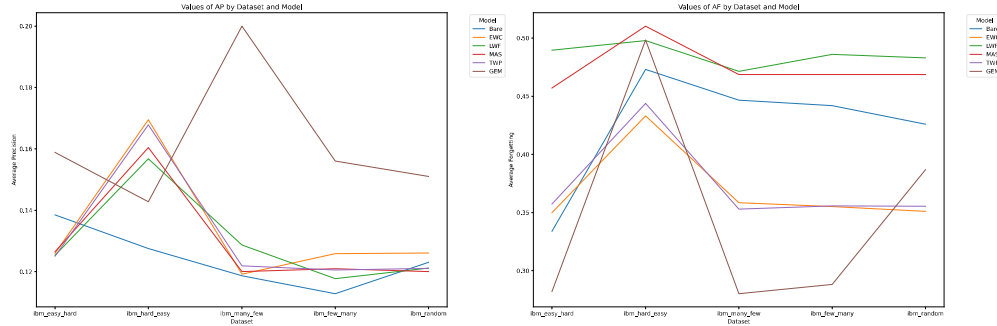


Figure 12: Global average of the average performance (left) and average forgetting (right) for the methods across the different permutations of the patterns.

seem to learn any knowledge from the new tasks. On the other hand, the models tend to overfit on the latest task for the IBM data set if epochs are set too high.

The complete collection of performance matrices is available as supplementary material online on Github⁴.

5.2 Order of Patterns

The analysis of the order of patterns is only relevant for the IBM data set. For the same architecture as before, we see in Figure 12 that the forgetting and precision is more or less stable across different pattern orders. Only for *hard to easy* we notice that the forgetting spikes for a couple of methods. Here, the average performance is also higher.

When aggregating on all experiments, as shown in Figure 13, we see that on average the order of the patterns does not seem to have a major impact on the performance nor on the forgetting. This is in line with previous studies [9, 43].

We notice that the boxes for EWC and TWP - which is based on EWC - go a bit higher in terms of performance in the hard-to-easy setting, than for the others. This might indicate that these regularisation-based methods perform a bit better in this setting, although differences are minor.

⁴<https://github.com/VerbekeLab>

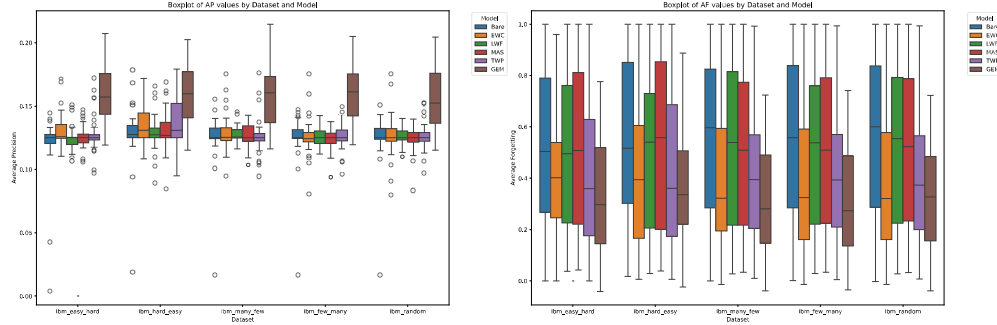


Figure 13: Boxplots of the average performance (left) and average forgetting (right) for the methods across the different permutations of the patterns.

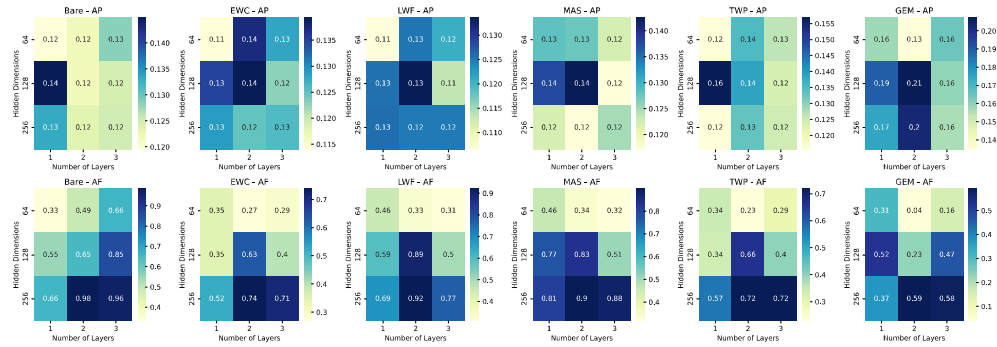


Figure 14: The average performance and average forgetting across different depths and widths of the GCN, when training for five epochs per task on the IBM data set.

5.3 Architecture of Backbone

In this section, we set out to answer **RQ3** by analysing the effect of the depth and width of the GCN model. The results in Figure 7 already show that the architecture of the GCN - both in terms of depth and width - has an impact on performance and forgetting. We provide heatmaps of the average performance and forgetting for all methods to have a detailed view on the results. The results for the IBM data set on the easy-to-hard data set for five epochs in Figure 14. The results for the Elliptic data set with seven and 49 tasks for five epochs in Figure 15 and Figure 16, respectively.

Although intuitively more layers should be better in terms of average performance, the results do not show a clear dominance of two or three layers over one layer in the GCN. On the other hand, we can clearly see that forgetting is more severe if the GCN has more parameters. Deeper and wider GCNs tend to overfit more on the latest task.

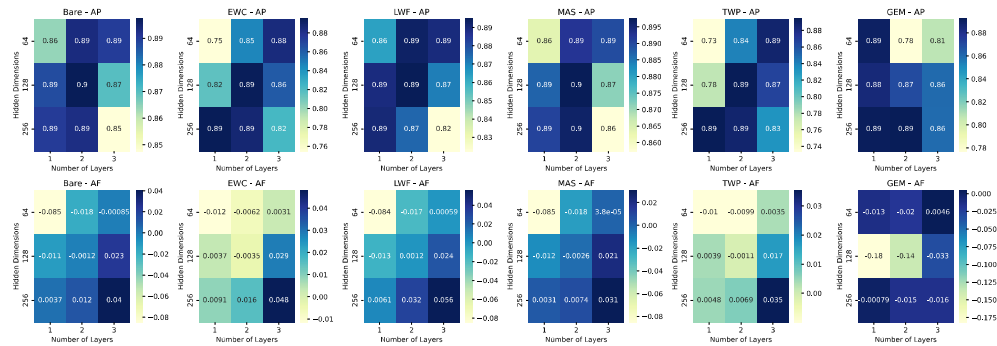


Figure 15: The average performance and average forgetting across different depths and widths of the GCN, when training for five epochs per task on the elliptic data set with seven tasks.

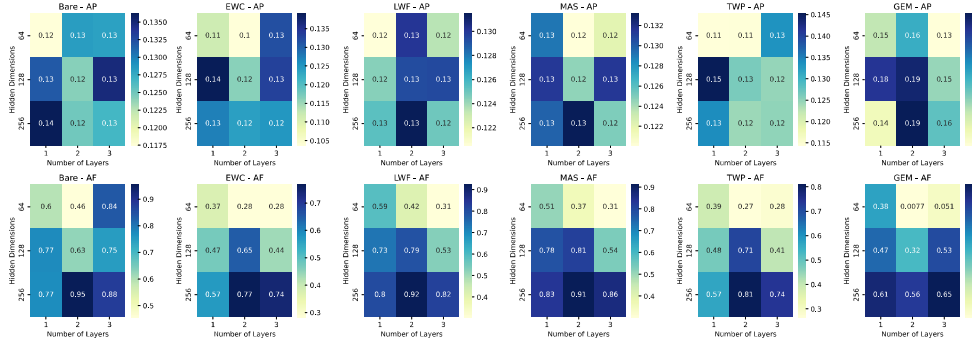


Figure 16: The average performance and average forgetting across different depths and widths of the GCN, when training for five epochs per task on the elliptic data set with 49 tasks.

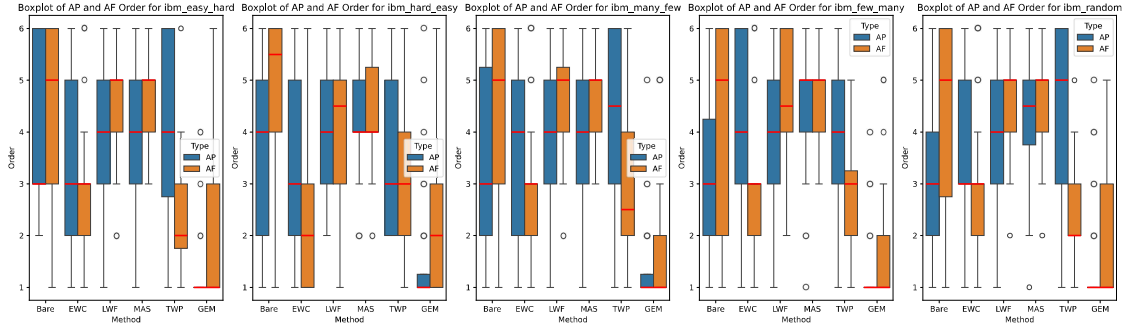


Figure 17: Box plot of the order of the different methods per permutation of the patterns on the IBM data set.

5.4 Continual Learning Method

To evaluate the effect of the continual learning method to answer **RQ4**, we first look at the performance matrices in Figure 8. Visually, it seems that GEM most often retains previous knowledge both for a low as well as high number of epochs, while also learning the new task.

We confirm this by assessing the box plots of the ranking of the models in Figure 17 and Figure 18 for the IBM and Elliptic data set, respectively. On average, GEM scores best both in terms of low average forgetting as well as high average performance for both data sets.

When looking at the IBM results, EWC and to a lesser extent TWP seems to also perform quite strongly. For the other methods, it seems that there is a trade-off between forgetting and performance.

For the Elliptic data set, the results are slightly different, here MAS performs well, while EWC is performing quite poorly. For the other methods, we again see a trade-off between forgetting and performance.

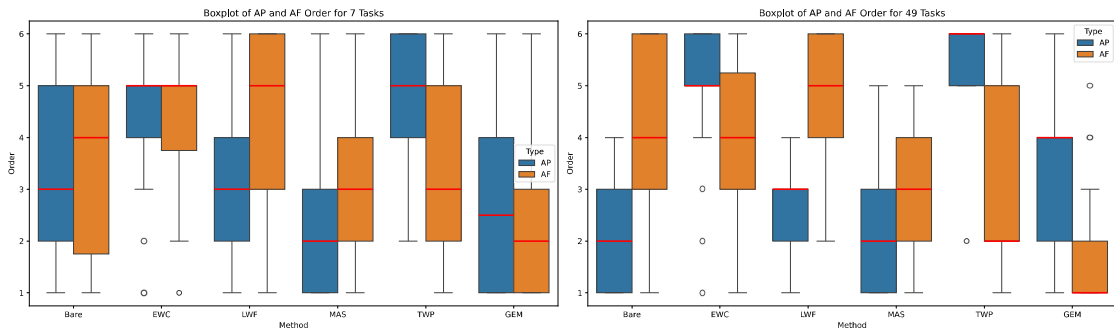


Figure 18: Box plot of the order of the different methods for the different number of tasks on the elliptic data set.

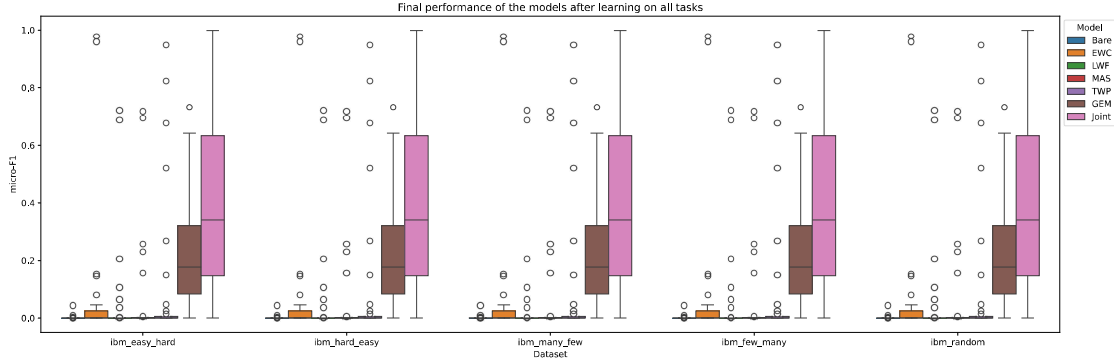


Figure 19: Box plot of the performance of the final model on all test data for the IBM data set.

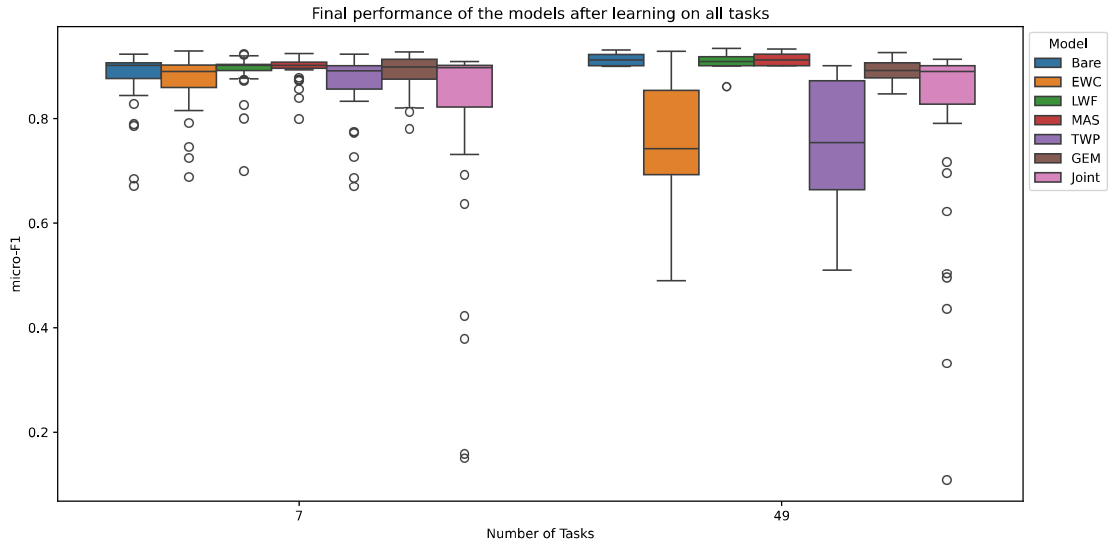


Figure 20: Boxplot of the performance of the final model on all test data for the IBM data set.

An observation made for all iterations is that continual learning methods seem to result in better AML detection methods than the bare and joint model. Looking at the performance matrices of the joint model, we see strong performance for the first task, concerning the majority class, but low performance on the other tasks. This results from the extreme class imbalance in AML. The joint GCN model seems to learn to always predict the majority class when minimising the loss function.

We also consider the performance of the model on the full test set, after seeing all tasks. The results on the IBM data set are given in Figure 19. As expected, the joint model outperforms all other models, since it was trained on all data. We also see that GEM performs better than the other continual learning and bare models. This is due to the combination of high average performance and low average forgetting.

Similar results are provided for the Elliptic data set in Figure 20. Here, the picture is less pronounced than before. We see consistently strong results for Bare, LwF, MAS and GEM. Surprisingly, the joint model seems to perform slightly worse than these models. Given that it is trained on all data simultaneously, the joint model might suffer from the inclusion of the sudden closure of a dark market at time step 43 [63].

6 Conclusion

Continual learning is essential in AML because (1) millions of transactions need to be monitored continuously, (2) fraud tactics are constantly evolving, causing underlying data distributions to shift, and (3) regulatory constraints often limit the amount of historical data that can be stored. Therefore, this work addresses four key research questions.

We started with reviewing the current state of the continual graph learning literature for AML (RQ1). We conclude that despite its ability to tackle these challenges, continual learning for AML has received limited interest in the scientific literature.

To expand on the current body of knowledge, we present the results of a comprehensive experiment on continual graph learning on two AML data sets. We presented experiments for node and edge classification. We investigated the effect of the hyperparameters including the task order (RQ2), the effect of the GNN architecture (RQ3), and the different continual learning methods (RQ4).

We conclude that increasing the number of epochs per task too much may lead to overfitting on the present task, and hence to forgetting. With regard to the task order, our experiments are in line with previous work and confirm that there is no significant effect of the task order on performance.

Based on the experimental results, we conclude that wide models are more prone to forgetting, and a balance needs to be struck between capturing longer money laundering chains and avoiding over-smoothing and forgetting when setting the depth of the GNN.

Across the experiments, GEM performed well with minimal forgetting. This indicates that replay methods are best when it comes to AML. As noted, their application in practice might be hindered by regulations limiting the storage of transaction data.

A surprising result is obtained regarding the joint model. The experiments show that continual learning methods are better at allowing the model to learn the different fraud patterns. When provided with all data, the joint model learns to consistently predict the majority class.

Based on the presented literature review and experimental evaluation, we identify a series of directions for future work. First of all, a deeper analysis is needed on the specific challenges in fraud detection and how these can be addressed by continual learning methods. Future research should investigate the effect of rotating between patterns, and the effect of having periods with no fraud cases, in a continual learning setting.

Second, the BeGIn framework should be extended to incorporate domain-incremental learning for edge classification, needed for extended analysis of the IBM data set, and to include more backbone architectures. This would result in extended experiments to complement the analysis done in this work.

Finally, some of the problems present for AML are also present in other domains, especially the high class imbalance. Qualitative and quantitative research on the interplay between the degree of imbalance and the performance of continual learning is still lacking.

Author Contributions

Bruno Deprez: Conceptualization, Data Curation, Methodology, Software, Visualization, Writing – Original Draft Preparation. **Wei Wei:** Conceptualization, Methodology, Software, Writing – Review & Editing. **Wouter Verbeke:** Conceptualization, Supervision, Writing – Review & Editing. **Bart Baesens:** Conceptualization, Supervision, Writing – Review & Editing. **Kevin Mets:** Conceptualization, Methodology, Supervision, Writing – Review & Editing. **Tim Verdonck:** Conceptualization, Methodology, Supervision, Writing – Review & Editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Research Foundation – Flanders (FWO research project 1SHEN24N), by the BNP Paribas Fortis Chair in Fraud Analytics, and by the Flemish Government under the “Onderzoeksprogramma Artificiële Intelligentie (AI) Vlaanderen” programme. The resources and services used in this work were provided by the VSC (Flemish Supercomputer Center), funded by the Research Foundation - Flanders (FWO) and the Flemish Government.

References

- [1] Muhammad Abulaish, Nesar Ahmad Wasi, and Shachi Sharma. The role of lifelong machine learning in bridging the gap between human and machine learning: A scientometric analysis. *WIREs Data Mining and Knowledge Discovery*, 14(2):e1526, 2024. doi:<https://doi.org/10.1002/widm.1526>. URL <https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/widm.1526>.
- [2] Ismail Alarab, Simant Prakoonwit, and Mohamed Ikbal Nacer. Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain. In *Proceedings of the 2020 5th International Conference on Machine Learning Technologies, ICMLT '20*, page 23–27, Beijing, China, 2020. Association for Computing Machinery. ISBN 9781450377645. doi:[10.1145/3409073.3409080](https://doi.org/10.1145/3409073.3409080). URL <https://doi.org/10.1145/3409073.3409080>.
- [3] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.
- [4] Erik Altman, Jovan Blanuša, Luc von Niederhäusern, Beni Egressy, Andreea Anghel, and Kubilay Atasü. Realistic synthetic financial transactions for anti-money laundering models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 29851–29874. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/5f38404edff6f3f642d6fa5892479c42-Paper-Datasets_and_Benchmarks.pdf.
- [5] Bart Baesens, Veronique Van Vlasselaer, and Wouter Verbeke. *Fraud analytics using descriptive, predictive, and social network techniques: a guide to data science for fraud detection*. John Wiley & Sons, Inc, 2015. ISBN 9781119133124. doi:[10.1002/9781119146841](https://doi.org/10.1002/9781119146841).
- [6] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML '09*, page 41–48, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605585161. doi:[10.1145/1553374.1553380](https://doi.org/10.1145/1553374.1553380). URL <https://doi.org/10.1145/1553374.1553380>.
- [7] Mário Cardoso, Pedro Saleiro, and Pedro Bizarro. Laundrograph: Self-supervised graph representation learning for anti-money laundering. In *Proceedings of the Third ACM International Conference on AI in Finance, ICAIF '22*, pages 130–138, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393768. doi:[10.1145/3533271.3561727](https://doi.org/10.1145/3533271.3561727). URL <https://doi.org/10.1145/3533271.3561727>.
- [8] Antonio Carta, Andrea Cossu, Federico Errica, and Davide Bacciu. Catastrophic forgetting in deep graph networks: an introductory benchmark for graph classification, 2021. URL <https://arxiv.org/abs/2103.11750>.
- [9] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2022. doi:[10.1109/TPAMI.2021.3057446](https://doi.org/10.1109/TPAMI.2021.3057446).
- [10] Bruno Deprez, Toon Vanderschueren, Bart Baesens, Tim Verdonck, and Wouter Verbeke. Network analytics for anti-money laundering – a systematic literature review and experimental evaluation, 2024. URL <https://arxiv.org/abs/2405.19383>.
- [11] Bruno Deprez, Félix Vandervorst, Wouter Verbeke, Tim Verdonck, and Bart Baesens. Network analytics for insurance fraud detection: a critical case study. *European Actuarial Journal*, 14(3):965–990, 2024. doi:[10.1007/s13385-024-00384-6](https://doi.org/10.1007/s13385-024-00384-6). URL <https://doi.org/10.1007/s13385-024-00384-6>.
- [12] Béni Egressy, Luc von Niederhäusern, Jovan Blanuša, Erik Altman, Roger Wattenhofer, and Kubilay Atasü. Provably powerful graph neural networks for directed multigraphs. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(10):11838–11846, Mar. 2024. doi:[10.1609/aaai.v38i10.29069](https://doi.org/10.1609/aaai.v38i10.29069). URL <https://ojs.aaai.org/index.php/AAAI/article/view/29069>.
- [13] Elliptic. Elliptic. www.elliptic.co. Accessed: 2024-01-31.
- [14] Falih Gozi Febrinanto, Feng Xia, Kristen Moore, Chandra Thapa, and Charu Aggarwal. Graph lifelong learning: A survey. *IEEE Computational Intelligence Magazine*, 18(1):32–51, 2023. doi:[10.1109/MCI.2022.3222049](https://doi.org/10.1109/MCI.2022.3222049).
- [15] Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 2025/02/25 1999. doi:[10.1016/S1364-6613\(99\)01294-2](https://doi.org/10.1016/S1364-6613(99)01294-2). URL [https://doi.org/10.1016/S1364-6613\(99\)01294-2](https://doi.org/10.1016/S1364-6613(99)01294-2).
- [16] Ian J. Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. An empirical investigation of catastrophic forgetting in gradient-based neural networks, 2015. URL <https://arxiv.org/abs/1312.6211>.
- [17] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural*

- Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7e9bea9-Paper.pdf.
- [18] Hamed Hemati, Marco Schreyer, and Damian Borth. Continual learning for unsupervised anomaly detection in continuous auditing of financial accounting data, 2022. URL <https://arxiv.org/abs/2112.13215>.
- [19] Chengxiang Jin, Jie Jin, Jiajun Zhou, Jiajing Wu, and Qi Xuan. Heterogeneous feature augmentation for ponzi detection in ethereum. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 69(9):3919–3923, 2022. doi:[10.1109/TCSII.2022.3177898](https://doi.org/10.1109/TCSII.2022.3177898).
- [20] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks, 2017.
- [21] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi:[10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114). URL <https://www.pnas.org/doi/abs/10.1073/pnas.1611835114>.
- [22] Jihoon Ko, Shinhwan Kang, Taehyung Kwon, Heechan Moon, and Kijung Shin. Begin: Extensive benchmark scenarios and an easy-to-use framework for graph continual learning, 2024. URL <https://arxiv.org/abs/2211.14568>.
- [23] B. Leblot, W. Siblini, G.M. Paldino, Y.-A. Le Borgne, F. Oblé, and G. Bontempi. Assessment of catastrophic forgetting in continual credit card fraud detection. *Expert Systems with Applications*, 249:123445, 2024. ISSN 0957-4174. doi:<https://doi.org/10.1016/j.eswa.2024.123445>. URL <https://www.sciencedirect.com/science/article/pii/S0957417424003105>.
- [24] Sebastian Lee, Sebastian Goldt, and Andrew Saxe. Continual learning in the teacher-student setup: Impact of task similarity. In *International Conference on Machine Learning*, pages 6109–6119. PMLR, 2021.
- [25] Michael Levi and Peter Reuter. Money laundering. *Crime and justice*, 34(1):289–375, 2006. doi:[10.1086/501508](https://doi.org/10.1086/501508).
- [26] An Li, Zhongshuai Wang, Minghao Yu, and Di Chen. Blockchain abnormal transaction detection method based on weighted sampling neighborhood nodes. In *2022 3rd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*, pages 746–752. IEEE, 2022. doi:[10.1109/ICBAIE56435.2022.9985815](https://doi.org/10.1109/ICBAIE56435.2022.9985815).
- [27] Qimai Li, Zhichao Han, and Xiao-ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), Apr. 2018. doi:[10.1609/aaai.v32i1.11604](https://doi.org/10.1609/aaai.v32i1.11604). URL <https://ojs.aaai.org/index.php/AAAI/article/view/11604>.
- [28] Yujie Li, Yuxuan Yang, Xin Yang, Qiang Gao, and Fan Zhou. Forgetting prevention for cross-regional fraud detection with heterogeneous trade graph, 2022. URL <https://arxiv.org/abs/2204.10085>.
- [29] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2018. doi:[10.1109/TPAMI.2017.2773081](https://doi.org/10.1109/TPAMI.2017.2773081).
- [30] Ziyu Li, Yanmei Zhang, Qian Wang, and Shiping Chen. Transactional network analysis and money laundering behavior identification of central bank digital currency of china. *Journal of Social Computing*, 3(3):219–230, 2022. doi:[10.23919/JSC.2022.0011](https://doi.org/10.23919/JSC.2022.0011).
- [31] J. Lian, K. Choi, B. Veeramani, A. Hu, S. Murli, L. Freeman, E. Bowen, and X. Deng. Continual learning and its industrial applications: A selective review. *WIREs Data Mining and Knowledge Discovery*, 14(6):e1558, 2024. doi:<https://doi.org/10.1002/widm.1558>. URL <https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/widm.1558>.
- [32] Huihui Liu, Yiding Yang, and Xinchao Wang. Overcoming catastrophic forgetting in graph neural networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(10):8653–8661, May 2021. doi:[10.1609/aaai.v35i10.17049](https://doi.org/10.1609/aaai.v35i10.17049). URL <https://ojs.aaai.org/index.php/AAAI/article/view/17049>.
- [33] Yilun Liu, Ruihong Qiu, and Zi Huang. Cat: Balanced continual graph learning with graph condensation. In *2023 IEEE International Conference on Data Mining (ICDM)*, pages 1157–1162, 2023. doi:[10.1109/ICDM58522.2023.00141](https://doi.org/10.1109/ICDM58522.2023.00141).
- [34] Wai Weng Lo, Gayan K. Kulatilleke, Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. Inspection-I: self-supervised gnn node embeddings for money laundering detection in bitcoin. *Applied Intelligence*, 53(16):19406–19417, 2023. doi:[10.1007/s10489-023-04504-9](https://doi.org/10.1007/s10489-023-04504-9). URL <https://doi.org/10.1007/s10489-023-04504-9>.
- [35] David Lopez-Paz and Marc' Aurelio Ranzato. Gradient episodic memory for continual learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/f87522788a2be2d171666752f97ddeb-Paper.pdf.

- [36] Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7765–7773, 2018. doi:[10.1109/CVPR.2018.00810](https://doi.org/10.1109/CVPR.2018.00810).
- [37] Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In Vittorio Ferrari, Martial Hebert, Cristian Sminchisescu, and Yair Weiss, editors, *Computer Vision – ECCV 2018*, pages 72–88, Cham, 2018. Springer International Publishing. ISBN 978-3-030-01225-0.
- [38] Nicolas Y. Masse, Gregory D. Grant, and David J. Freedman. Alleviating catastrophic forgetting using context-dependent gating and synaptic stabilization. *Proceedings of the National Academy of Sciences*, 115(44): E10467–E10475, 2018. doi:[10.1073/pnas.1803839115](https://doi.org/10.1073/pnas.1803839115). URL <https://www.pnas.org/doi/abs/10.1073/pnas.1803839115>.
- [39] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. volume 24 of *Psychology of Learning and Motivation*, pages 109–165. Academic Press, 1989. doi:[https://doi.org/10.1016/S0079-7421\(08\)60536-8](https://doi.org/10.1016/S0079-7421(08)60536-8). URL <https://www.sciencedirect.com/science/article/pii/S0079742108605368>.
- [40] Seyed Iman Mirzadeh, Arslan Chaudhry, Dong Yin, Timothy Nguyen, Razvan Pascanu, Dilan Gorur, and Mehrdad Farajtabar. Architecture matters in continual learning, 2022. URL <https://arxiv.org/abs/2202.00275>.
- [41] Anuraj Mohan, Karthika P. V., Parvathi Sankar, K. Maya Manohar, and Amala Peter. Improving anti-money laundering in bitcoin using evolving graph convolutions and deep neural decision forest. *Data Technologies and Applications*, 57(3):313–329, 2023. doi:[10.1108/DTA-06-2021-0167](https://doi.org/10.1108/DTA-06-2021-0167). URL <https://doi.org/10.1108/DTA-06-2021-0167>.
- [42] Soroor Motie and Bijan Raahemi. Financial fraud detection using graph neural networks: A systematic review. *Expert Systems with Applications*, 240:122156, 2024. ISSN 0957-4174. doi:<https://doi.org/10.1016/j.eswa.2023.122156>. URL <https://www.sciencedirect.com/science/article/pii/S0957417423026581>.
- [43] Cuong V. Nguyen, Alessandro Achille, Michael Lam, Tal Hassner, Vijay Mahadevan, and Stefano Soatto. Toward understanding catastrophic forgetting in continual learning, 2019. URL <https://arxiv.org/abs/1908.01091>.
- [44] María Óskarsdóttir, Waqas Ahmed, Katrien Antonio, Bart Baesens, Rémi Dendievel, Tom Donas, and Tom Reynkens. Social network analytics for supervised fraud detection in insurance. *Risk Analysis*, 42(8):1872–1890, 2022. doi:[10.1111/risa.13693](https://doi.org/10.1111/risa.13693).
- [45] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019. ISSN 0893-6080. doi:<https://doi.org/10.1016/j.neunet.2019.01.012>. URL <https://www.sciencedirect.com/science/article/pii/S0893608019300231>.
- [46] Massimo Perini, Giorgia Ramponi, Paris Carbone, and Vasiliki Kalavri. Learning on streaming graphs with experience replay. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, SAC '22*, page 470–478, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450387132. doi:[10.1145/3477314.3507113](https://doi.org/10.1145/3477314.3507113). URL <https://doi.org/10.1145/3477314.3507113>.
- [47] M.I. Pramanik, Raymond Y.K. Lau, Wei T. Yue, Yunming Ye, and Chunping Li. Big data analytics for security and criminal investigations. *WIREs Data Mining and Knowledge Discovery*, 7(4):e1208, 2017. doi:<https://doi.org/10.1002/widm.1208>. URL <https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/widm.1208>.
- [48] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.
- [49] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/fa7cdfad1a5aaf8370ebeda47a1ffc3-Paper.pdf.
- [50] Franco Scarselli, Sweah Liang Yong, Marco Gori, Markus Hagenbuchner, Ah Chung Tsoi, and Marco Maggini. Graph neural networks for ranking web pages. In *The 2005 IEEE/WIC/ACM International Conference on Web Intelligence (WI'05)*, pages 666–672. IEEE, 2005. doi:[10.1109/WI.2005.67](https://doi.org/10.1109/WI.2005.67).

- [51] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008. doi:[10.1109/TNN.2008.2005605](https://doi.org/10.1109/TNN.2008.2005605).
- [52] Joan Serra, Didac Suris, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4548–4557. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/serra18a.html>.
- [53] Kai Sun, Kun Meng, and Ziqiang Zheng. Game-bc: A graph attention model for exploring bitcoin crime. In *2022 6th International Symposium on Computer Science and Intelligent Control (ISCSIC)*, pages 342–346. IEEE, 2022. doi:[10.1109/ISCSIC57216.2022.00077](https://doi.org/10.1109/ISCSIC57216.2022.00077).
- [54] United Nations Office on Drugs and Crime (UNODC). Money laundering. <https://www.unodc.org/unodc/en/money-laundering/overview.html>. Accessed: 2023-04-07.
- [55] Rafaël Van Belle, Sandra Mitrović, and Jochen De Weerd. Representation learning in graphs for credit card fraud detection. In Valerio Bitetta, Ilaria Bordino, Andrea Ferretti, Francesco Gullo, Stefano Pascolutti, and Giovanni Ponti, editors, *Mining Data for Financial Applications*, pages 32–46, Cham, 2020. Springer International Publishing. ISBN 978-3-030-37720-5.
- [56] Rafaël Van Belle and Jochen De Weerd. Shine: A scalable heterogeneous inductive graph neural network for large imbalanced datasets. *IEEE Transactions on Knowledge and Data Engineering*, 36(9):4904–4915, 2024. doi:[10.1109/TKDE.2024.3381240](https://doi.org/10.1109/TKDE.2024.3381240).
- [57] Rafaël Van Belle, Charles Van Damme, Hendrik Tytgat, and Jochen De Weerd. Inductive graph representation learning for fraud detection. *Expert Systems with Applications*, 193:116463, 2022. ISSN 0957-4174. doi:<https://doi.org/10.1016/j.eswa.2021.116463>. URL <https://www.sciencedirect.com/science/article/pii/S0957417421017449>.
- [58] Gido M. van de Ven, Tinne Tuytelaars, and Andreas S. Tolias. Three types of incremental learning. *Nature Machine Intelligence*, 4(12):1185–1197, 2022. doi:[10.1038/s42256-022-00568-3](https://doi.org/10.1038/s42256-022-00568-3). URL <https://doi.org/10.1038/s42256-022-00568-3>.
- [59] Véronique Van Vlasselaer, Cristián Bravo, Olivier Caelen, Tina Eliassi-Rad, Leman Akoglu, Monique Snoeck, and Bart Baesens. Apate: A novel approach for automated credit card transaction fraud detection using network-based extensions. *Decision Support Systems*, 75:38–48, 2015. ISSN 0167-9236. doi:<https://doi.org/10.1016/j.dss.2015.04.013>. URL <https://www.sciencedirect.com/science/article/pii/S0167923615000846>.
- [60] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks, 2018.
- [61] Junshan Wang, Guojie Song, Yi Wu, and Liang Wang. Streaming graph neural networks via continual learning. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management, CIKM '20*, page 1515–1524, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450368599. doi:[10.1145/3340531.3411963](https://doi.org/10.1145/3340531.3411963). URL <https://doi.org/10.1145/3340531.3411963>.
- [62] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(8):5362–5383, 2024. doi:[10.1109/TPAMI.2024.3367329](https://doi.org/10.1109/TPAMI.2024.3367329).
- [63] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics, 2019. URL <https://arxiv.org/abs/1908.02591>.
- [64] Wei Wei, Tom De Schepper, and Kevin Mets. Benchmarking sensitivity of continual graph learning for skeleton-based action recognition. In *Proceedings of the 19th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*, page 639–651. SCITEPRESS - Science and Technology Publications, 2024. doi:[10.5220/0012394400003660](https://doi.org/10.5220/0012394400003660). URL <http://dx.doi.org/10.5220/0012394400003660>.
- [65] Wei Wei, Tom De Schepper, and Kevin Mets. Dataset condensation with latent quantile matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 7703–7712, June 2024.
- [66] Pingfan Xia, Zhiwei Ni, Hongwang Xiao, Xuhui Zhu, and Peng Peng. A novel spatiotemporal prediction approach based on graph convolution neural networks and long short-term memory for money laundering fraud. *Arabian Journal for Science and Engineering*, 47(2):1921–1937, 2022. doi:[10.1007/s13369-021-06116-2](https://doi.org/10.1007/s13369-021-06116-2). URL <https://doi.org/10.1007/s13369-021-06116-2>.
- [67] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks?, 2019.

- [68] Qiao Yuan, Sheng-Wei Guan, Pin Ni, Tianlun Luo, Ka Lok Man, Prudence Wong, and Victor Chang. Continual graph learning: A survey, 2023. URL <https://arxiv.org/abs/2301.12230>.
- [69] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3987–3995. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/zenke17a.html>.
- [70] Peiyan Zhang, Yuchen Yan, Chaozhuo Li, Senzhang Wang, Xing Xie, Guojie Song, and Sunghun Kim. Continual learning on dynamic graphs via parameter isolation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '23, page 601–611, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394086. doi:[10.1145/3539618.3591652](https://doi.org/10.1145/3539618.3591652). URL <https://doi.org/10.1145/3539618.3591652>.
- [71] Rui Zhang, Dawei Cheng, Jie Yang, Yi Ouyang, Xian Wu, Yefeng Zheng, and Changjun Jiang. Pre-trained online contrastive learning for insurance fraud detection. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(20):22511–22519, Mar. 2024. doi:[10.1609/aaai.v38i20.30259](https://doi.org/10.1609/aaai.v38i20.30259). URL <https://ojs.aaai.org/index.php/AAAI/article/view/30259>.
- [72] Shilei Zhang, Toyotaro Suzumura, and Li Zhang. Dyngraphtrans: Dynamic graph embedding via modified universal transformer networks for financial transaction data. In *2021 IEEE International Conference on Smart Data Services (SMDS)*, pages 184–191, 2021. doi:[10.1109/SMDS53860.2021.00032](https://doi.org/10.1109/SMDS53860.2021.00032).
- [73] Xikun Zhang, Dongjin Song, and Dacheng Tao. Cglb: Benchmark tasks for continual graph learning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 13006–13021. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/548a41b9cac6f50dccf7e63e9e1b1b9b-Paper-Datasets_and_Benchmarks.pdf.
- [74] Xikun Zhang, Dongjin Song, and Dacheng Tao. Continual learning on graphs: Challenges, solutions, and opportunities, 2024. URL <https://arxiv.org/abs/2402.11565>.
- [75] Tianqi Zhao, Alan Hanjalic, and Megha Khosla. AGALE: A graph-aware continual learning evaluation framework. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=xDTKRLyaNN>.
- [76] Xin Zheng, Miao Zhang, Chunyang Chen, Quoc Viet Hung Nguyen, Xingquan Zhu, and Shirui Pan. Structure-free graph condensation: From large-scale graphs to condensed graph-free data. *Advances in Neural Information Processing Systems*, 36:6026–6047, 2023.
- [77] Da-Wei Zhou, Qi-Wei Wang, Zhi-Hong Qi, Han-Jia Ye, De-Chuan Zhan, and Ziwei Liu. Class-incremental learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):9851–9873, 2024. doi:[10.1109/TPAMI.2024.3429383](https://doi.org/10.1109/TPAMI.2024.3429383).
- [78] Fan Zhou and Chengtai Cao. Overcoming catastrophic forgetting in graph neural networks with experience replay. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(5):4714–4722, May 2021. doi:[10.1609/aaai.v35i5.16602](https://doi.org/10.1609/aaai.v35i5.16602). URL <https://ojs.aaai.org/index.php/AAAI/article/view/16602>.